

Appendix D

Additional Case Studies

These four case studies accompany *Chapter 16* and are offered online with the book. They continue the chapter's numbering from Case study 5: two open-source personal-assistant ecosystems, two shipping coding agents, a teaching composite of the software lifecycle, and an illustrative design case in regulated finance. Like the four in the chapter, each is read against the same five questions:

1. **Where does authority sit?** Which component may act without asking, and what bounds it.
2. **What evidence crosses each boundary?** What one stage hands the next, and in what form.
3. **How are effects contained?** What stops a wrong action from reaching the systems it would change.
4. **Which bottleneck binds?** The boundary that commits the most downstream work, or the resource everything waits on.
5. **What decision does an evaluation control?** Which measure can hold, roll back, or escalate a change, and which only fills a dashboard.

Each case names its baseline, the simpler system it has to beat, and closes on lessons that answer these questions and say how far the answers travel. The last case runs the questions forward on a workflow no case has deployed, and *Chapter 16's What the cases show together* reads across all eight. The chapter's evidence rules hold here too: "reported" never silently becomes "measured", and a case that leans on a benchmark says what changed between the benchmark and use.

Case study 5: Personal AI assistants, OpenClaw and Hermes as two bets on the same problem

This case sets two open-source personal assistants, OpenClaw and Hermes, side by side, each betting on a different part of the problem as the hard one.

The problem

The personal AI assistant is one of the oldest goals in consumer software, and language models made a general one plausible for the first time: every earlier generation collapsed into a scripted menu that broke the moment the user stepped off the script. A language model lets one general assistant hold an open-ended conversation, call tools, and write its own notes, and that generality makes it a hard architecture problem. Overall usefulness has no oracle beyond whether the operator keeps using it, much as research lacked a cheap acceptance test in *Case study 2* in *Chapter 16*. Many individual obligations do have one: a booking is right or wrong, an action was authorized or not, and a message reached the intended recipient or someone else. The assistant runs on one operator's own machine with that person's credentials, answers on the messaging channels they already use (WhatsApp, Telegram, Slack, Discord, Signal), and extends itself by loading *skills*, procedure files that teach it to do something new. Skills are how the assistant grows, and they arrive as installable code, often written by strangers, so deciding who may add that code and what it can reach is as much the design problem as the dialogue. A campaign named ClawHavoc proved the point in early 2026 by seeding ClawHub, OpenClaw's skill marketplace, with malicious skills.

Two open-source projects made opposite bets about which part of that problem is hard. OpenClaw, a TypeScript system with twenty-five-plus channel integrations, bets the hard problem is *routing and control*: getting the right message to the right agent with the right permissions [OpenClaw, OpenClaw]. Hermes, a Python system from Nous Research, bets the hard problem is *memory and self-improvement*: an agent that learns the operator and rewrites its own behavior over time [Hermes, Hermes Agent]. An independent technical comparison called them "two valid bets about what the hard problem of personal AI actually is" [Trilogy, Technical deep dive]. The baseline both must beat is the same model in one chat loop with the operator's tools and no persistent memory, which handles each conversation well and forgets it. Neither project reports a comparison against it, so what follows reads their architecture and not their effectiveness.

Architecture: Gateway-first versus learning-first

OpenClaw is organized around a single long-lived **Gateway** process that acts as a control plane. Every inbound message flows through it, and through one routing decision, before any reasoning happens. That decision is deliberately deterministic and modelless:

`resolveAgentRoute()` walks a tiered binding list (by peer, then group and role, then team, then account, then channel, falling back to a default) and the first, most specific match wins. The agent that handles a turn is therefore a pure function of who sent it and where, read from cached configuration. CAIPE reached the same place in *Case study 1* in *Chapter 16*: the cheapest router is a person or a configuration, with no tokens spent and no routing hallucination to debug. Each named agent is a logical identity with its own workspace directory, SQLite database, and session history; isolation between agents is by filesystem and session key, not by process. The routing topology is a star, one Gateway dispatching to named agents, but delegation beneath it has deepened. The project once listed manager-of-managers trees among the designs it rejects, and now lets a subagent orchestrate children of its own several levels down. A child reports back by injecting a typed `task_completion` event into its parent's session through the same Gateway dispatch, so a delegated result travels the routed path every other message takes.

A binding table is the operator's only while the model cannot write it, and OpenClaw learned that on its own configuration. Its agent-facing gateway tool once let a model patch the Gateway's configuration behind a hand-maintained denylist of sensitive paths. As the schema grew, the denylist fell behind, leaving settings that govern command execution, credential forwarding, telemetry endpoints, and operator policy writable by a compromised or prompt-injected model [OpenClaw, Config Mutation Advisory]. The fix replaced the denylist in April 2026 with a fail-closed allowlist of a few prompt and model settings, and in July a later change removed configuration writes from the agent's tool altogether, sending persistent changes through a human-approved path instead.

Hermes inverts the emphasis. Its durable core is a single `AIAgent` running a standard reason-act tool loop, and multi-agent structure appears only when work demands it. The `delegate_task` tool spawns subagents, which by default run in the background and report back as a new message. Each takes one of two roles: a leaf is denied the delegation, memory, and messaging tools so it cannot recurse or write shared state, while an orchestrator keeps the delegation tool and may coordinate leaves of its own. A spawn-depth limit and a concurrent-children limit govern how large that tree grows, and an operator can raise either without bound. The project's postmortem of one long refactoring run with a raised depth limit shows where a deep tree spends: 1,394 sessions at an estimated \$19,302.59, of which \$11,159.76 went to writing prompt caches. All 332 delegation timeouts in that run fell in nested delegates [Hermes, Hermes Agent]. Routing collapses into session-keying: the gateway maps each conversation to its own long-lived `AIAgent`, and that one agent handles the whole thread. For work that outgrows one conversation, Hermes adds a Kanban board in SQLite from which separate Hermes processes claim tasks, each claim a compare-and-swap on the task's status so that exactly one worker wins. A `swarm` command writes a fixed graph onto that board: parallel

specialist workers, a verifier that waits for all of them, then a synthesizer, with the shared blackboard kept as structured comments on the root task. Coordination through the board is *Chapter 8's* stigmergy, and the compare-and-swap closes in the database the lost update that chapter lists among the concurrency anomalies of shared state. Both dispatch shallowly at the top, but their delegation defaults have parted: Hermes keeps delegation flat unless the operator opts in, and OpenClaw permits recursion out of the box.

Figure D.1 sets the two bets side by side, with the learning loops and the skill supply chain that the rest of the case examines.



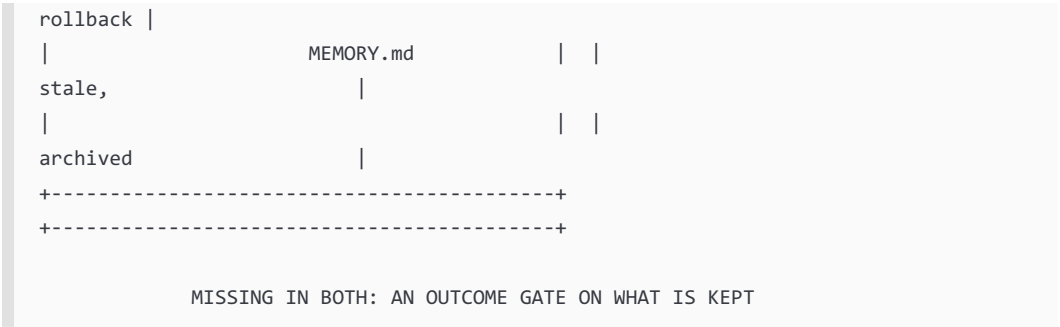


Figure D.1: OpenClaw and Hermes as two bets on the same problem. OpenClaw's Gateway routes each message through an operator-written binding table to a named agent with its own workspace and memory, whose subagents may nest, holds shell calls in an approval queue, and consolidates memory nightly into MEMORY.md. Hermes keys each conversation to one agent that delegates to leaves or orchestrators, while a background-review fork writes memory and skills and the Curator ages them. Third-party skills enter both through the same supply chain, and neither gates what it keeps on an outcome.

Design patterns and trade-offs

The systems converge on more than they advertise, and Table D.1 sets the differences beside the convergences. Its cells are a snapshot read at the two repositories on 21 September 2026 (OpenClaw 2026.9.5, Hermes v2026.9.21), and the defaults in them move from release to release.

Dimension	OpenClaw (gateway-first)	Hermes (learning-first)
Core bet	routing and control	memory and self-improvement
Topology	star: one Gateway routes to named agents	one agent that delegates; Kanban swarm on demand
Default spawn depth	5 (previously 1)	1, up to 10 concurrent children (no ceiling on either)
Memory	nightly "dreaming"; deep phase promotes to MEMORY.md at score ≥ 0.75 , ≥ 3 recalls, ≥ 3 distinct queries	bounded files (2,200 and 1,375 characters); review fork every 10 turns

Dimension	OpenClaw (gateway-first)	Hermes (learning-first)
Skill evolution	Skill Workshop; default auto edits skills directly, propose stages drafts	review fork every 10 tool iterations writes skills directly; staging opt-in
Skill upkeep	weekly collection review	Curator weekly: stale at 14 days unused, archived at 30; LLM merge opt-in
Change record	proposal hashes and rollback (propose mode); MEMORY.md preimage per rewrite	per-mutation ledger with before/after hashes and rollback (on)
Third-party skill intake	ClawHub scan with daily rescan; skills verify for exact versions	Skills Guard scan with a trust-tier install policy
Safety gate	runtime approval queue, policy attestation	approval patterns with hardline floors, the OS as the only boundary
Observability	bundled OpenTelemetry and Prometheus	opt-in Langfuse
Evaluation shipped	10 deterministic personal-agent scenarios on a mock model	regression probes for its own machinery

Table D.1: OpenClaw and Hermes, the same problem answered with opposite bets, as read on 21 September 2026

One convergence is easy to miss: both adopted the same open SKILL.md skill format, so the ecosystem standardized its periphery even as the cores diverged [Agent Skills]. The supply chain is shared literally, since Hermes can install skills straight from ClawHub. The format requires only a name and a description. None of its optional fields (a license, a compatibility note, free-form metadata, an experimental list of tools to pre-approve) records a publisher or signs the contents, so trust has to come from the registry that serves a skill and the client that loads it.

Safety and the honest boundary

Both projects are explicit that a personal assistant running with an operator's credentials is a different threat model from a multi-tenant service, and both are unusually candid about where the real boundary lies. OpenClaw makes the human-in-the-loop gate a first-class runtime object: an `ExecApprovalManager` suspends a shell tool call in a typed, expiring approval queue until the operator resolves it. A policy command can also record an attestation hash over the policy, the workspace, and the check's findings, then report drift when that state later changes. An assistant also acts when nobody has written to it, and those turns need a principal too. OpenClaw's heartbeat wakes the agent on a timer, and until March 2026 a heartbeat turn could inherit the owner's standing, which a critical advisory showed could carry a run past the sandbox [OpenClaw, Heartbeat Advisory]. The fix downgrades a wake whose content did not come from the owner, and two later advisories found wake sources the downgrade had missed: a background command's completion and a webhook. Hermes backs a pattern-based approval gate with hardline floors that block the most destructive commands even when approvals are off, and freezes its "yolo" auto-approve mode at import so a malicious skill cannot flip it. It pairs that gate with a blunt statement in its security documentation that "the only security boundary against an adversarial LLM is the operating system," directing operators to run the agent inside a container or a remote sandbox backend for any real isolation.

ClawHavoc is the field test of that candor, because no in-process approval queue stands between an operator and a package they chose to install. In early February 2026, Koi Security disclosed that 341 of the 2,857 skills then on ClawHub were malicious, 335 of them traceable to that one coordinated campaign [Koi Security, ClawHavoc]. The campaign's payload was mostly prose: a professional-looking "Prerequisites" section told the operator to download a password-protected archive, which antivirus scanners cannot open, or to paste an obfuscated command into a terminal that on macOS fetched a variant of the Atomic Stealer. A few of the other malicious skills hid a backdoor in code that ran only when the skill was used as intended. A mid-February update to the same disclosure raised the count to 824, more than double the original figure, on a marketplace that had itself grown past 10,700. The attack surface was the skill supply chain, not the conversation loop, and the payload ran with every permission the operator had.

Both projects have since hardened the intake. Within a week of the disclosure, OpenClaw began scanning every skill published to ClawHub with VirusTotal [OpenClaw, VirusTotal Partnership]. A benign verdict approves the skill, a suspicious one publishes it with a warning, and a malicious one blocks its download, while a daily rescan of every active skill catches one that turns malicious after approval. The announcement concedes that a skill whose natural-language instructions tell an agent to do harm will not match a virus signature, and ClawHavoc's prerequisites were exactly that. Installed ClawHub skills now record the registry

and version they came from, and `openclaw skills verify` asks the registry for the publisher, provenance, and scan state of that exact version. Hermes scans on install and weighs the verdict against the source's trust tier, so a caution-level finding installs from a trusted repository and is blocked from a community one. Its security policy calls the scanner a review aid and puts the boundary for third-party skills at operator review before install, since skills execute arbitrary Python at import time. The same reasoning leaves the scan of skills the agent writes for itself off by default: the agent could run the same code through its terminal tool, so the scan would mostly block prose that mentions risky keywords. Between them, the two projects have built *Chapter 14*'s admission gate and its rescan against a rug pull. Against an attack written as instructions, the most either gate can add is another model reading it. A skill the agent writes for itself lands where a downloaded one does, so how each system learns is a safety question too.

How each system tries to get better over time

Both systems want to serve one person better the longer they run, and both implement that as a write path that turns experience into durable artifacts.

OpenClaw models consolidation on sleep, and its memory plugin is *Chapter 7*'s dream pass as shipping code. A nightly sweep it calls *dreaming* runs three phases named after sleep: a light phase stages recent material, a REM phase reflects on recurring themes, and only the deep phase writes durable memory, promoting the strongest snippets into a `MEMORY.md` file. The agent's own retrieval history nominates what counts as strongest, since every memory search adds to a per-snippet recall count. A snippet is promoted only when a weighted score, led by relevance, frequency, and query diversity with a recency half-life, clears a threshold. It must also have been recalled a minimum number of times across a minimum number of distinct queries (Table D.1 gives the defaults). The query-diversity floor is the decision worth copying, since it stops one repeated question from promoting its own answer. The candidates that pass go to a tool-free model call that returns a list of operations against the current file (add, merge, supersede). The writer applies them only if the rewrite stays within a configured bound on how many prior entries it drops, cites every promoted candidate's source lines, and fits the file's budget. The previous `MEMORY.md` is stored before each rewrite. Candidates whose recorded provenance is untrusted or system-generated are removed before the model sees them, and snippets that were recalled into context are stripped from transcripts before ingestion, so the store cannot learn its own recalls back as new memory. At write time, the code reloads each candidate from its live source and refuses the write if that source changed during the operation, and it records the candidate's lineage beside the stored preimage. Those checks add the guards *Chapter 7*'s reconcile sketch leaves implicit: a taint gate on provenance, a bound on how much one pass may destroy, and a break in the loop through which a memory system would otherwise reinforce itself.

Hermes consolidates in-session rather than overnight. On a ten-step clock it spawns a background-review fork, a forked agent limited to the memory and skill tools plus read-only file access, which runs after the reply has already been sent. The fork's prompt pushes it hard to act: "most sessions produce at least one skill update," and a pass that saves nothing is "a missed learning opportunity". That fork writes facts into bounded memory files and, more ambitiously, creates and patches its own skills. Its tools are scoped to what triggered it: a review fired by the skill clock gets no memory tool, so that an unattended fork cannot act on a consolidation hint and delete memory entries no one reviewed. A separate **Curator** runs on a slower, roughly weekly clock and only after the agent has sat idle. It ages skills by a pure timestamp rule, from active to stale to archived, and does not read a zero usage count as staleness, since the counter may simply be new. An optional language-model pass, off by default, merges overlapping skills into broader umbrella skills, and for the same reason its prompt says to "judge overlap on content, not on use_count". The whole pass is bound by hard invariants: it touches only skills the agent itself wrote, refuses pinned, bundled, hub-installed, and external ones, fails closed when it cannot verify who wrote a skill, never deletes, and at most archives to a recoverable folder. A background review must also have loaded the exact skill it patches during the current review, so it cannot write over a version it has not read. Beneath it, every skill mutation from any path appends to a ledger with before-and-after content hashes, from which a single edit can be rolled back. The ledger and the Curator's aging rule ship as code much of the metadata *Chapter 13* asks every learned entry to carry: provenance, a rollback path, and staleness standing in for a time to live.

Both designs move learning off the hot path and protect the prompt prefix cache: OpenClaw consolidates offline, and Hermes freezes its memory snapshot into the system prompt and injects skills as user messages. Hermes even keeps the review fork's advertised tool list byte-identical to its parent's, so the cached prefix never shifts mid-session. Their signals of value are where they part: OpenClaw counts *use* directly, and Hermes' Curator decides on recency, plus content overlap when the merge pass is on. Both signals are cheap and need no supervision, which makes the loops practical, but use tracks relevance, and recency plus overlap tracks only age and similarity. Both projects also rely on a prompt instruction to keep only what worked. Hermes' review prompt forbids writing up a session's failed attempts as a reliable workflow, and OpenClaw's experience review, which runs on the same provider and model as the turn it reviews, captures only procedures that visibly worked before the operator interrupted. Each is a judgment by the model that did the work, read from the transcript it produced, and a reviewer that shares the worker's model shares its blind spots. Both projects treat persistence as a commit: OpenClaw checks where a candidate came from and whether its source held still, and Hermes checks who wrote the target and whether it was freshly read. Those checks guard the write, and neither project checks whether the kept entry improves behavior.

OpenClaw built such a check once: a "shadow trial" compared an answer with and without a candidate memory and labeled the candidate helpful, neutral, or harmful. It only ever ran as a report, and the project later deleted it as a feature with no supported product path. Both projects do ship tests. OpenClaw ships a personal-agent pack of ten deterministic scenarios on a synthetic channel and a mock model. They check that a reminder round-trips, a denied approval stops the action, a preference is recalled from the memory files, and a completion claim is backed by local evidence. Hermes ships regression probes for its own machinery, and both test the harness on fake users. Both already record the traces such a check would score, OpenClaw through bundled OpenTelemetry and Prometheus exporters and Hermes through an opt-in Langfuse integration and views over its session database.

Where the human enters the loop

A personal assistant has one evaluator who is always present, the operator, and the two projects first split on how that person's reactions become durable change. OpenClaw kept the operator in the write path: memory, user profile, and persona live in human-readable files (`MEMORY.md`, `USER.md`, `SOUL.md`) the operator can open and edit. Its Skill Workshop turns a generated skill into a *proposal*, a pending draft bound to the hash of the skill it changes, rescanned when it is applied, with rollback metadata written first. Hermes took the operator out of the write path and inferred the feedback instead. The operator corrects in conversation, and the background-review fork is prompted to treat corrections such as "stop doing X" as first-class signals and encode them as a step in the skill that governs that work.

The defaults have since converged on the Hermes side. OpenClaw's Workshop began with autonomous capture opt-in, then made its lifecycle approvals automatic, then made self-learning automatic by default. A background review now edits the Workshop's skills directly with ordinary file tools, without a proposal, a post-edit scan, or a rollback snapshot, and the proposal path survives as a propose mode an operator selects. Its reviewer now mines corrections too, treating a turn the operator interrupted as evidence because the wrong path and its correction are what is worth keeping. Hermes offers staged approval for memory and skill writes as an option, off by default, and a review surface where the operator can edit or delete what was learned after the fact. Both projects have moved the operator from before the write to after it: approval became a setting, and what stays on by default is a record the operator can inspect and roll back.

Results and lessons learned

Both projects drew hundreds of thousands of GitHub stars within months of release, Hermes on a near-weekly release cadence [OpenClaw, OpenClaw] [Hermes, Hermes Agent]. Stars measure developer interest, not deployment, quality, or reliability, but they do reflect real demand for self-hosted personal agents. The two bets are also complementary by construction: an assistant that dispatches a conversation to the wrong agent cannot benefit from good

memory, and one that dispatches correctly but forgets the operator stays at the baseline. The lessons that carry beyond the personal assistant follow the order a message meets the system: who handles it, what code it can reach, and what the system keeps afterward.

Lesson 1: Commit the route with a lookup the operator owns, and keep it where the model cannot write. A wrong binding sends a conversation to the wrong agent's workspace and memory, and no downstream quality recovers from that, so the route is the bottleneck that commits every later step. OpenClaw's own advisory states the premise its binding table rests on: the model is not a trusted principal. The path its configuration tool took, from denylist to allowlist to no writes at all, is the transferable part: a denylist decays as the thing it guards gains fields, and an allowlist fails closed as it does. Where a model has to route by intent, as neither project does, that routing decision is the first one to evaluate.

Lesson 2: Admit the agent's own skills through the gate a stranger's must pass, and put the real boundary in the operating system. Both systems are gentler with a skill the agent writes than with one a stranger publishes, though the two run with the same permissions. Hermes scans an agent-written skill only when the operator turns that scan on, and even then installs it on a caution-level finding that would block a community skill, and on a dangerous one only returns an error the agent can retry. OpenClaw's default mode edits the agent's skills with no post-edit scan and no rollback snapshot, and tells operators to keep backups. Record the exact source, version, and content hash of every skill, as ClawHub installs now do, so a compromised one can be revoked everywhere it landed. That record is *Chapter 14's* AI bill of materials, and revoking from it is that chapter's kill switch. Then run the agent inside an operating-system boundary sized to what the operator can afford to lose, one that encloses a single trust domain: OpenClaw's own multi-user documentation says anyone who can operate an agent can make it do anything the agent can do.

Lesson 3: Count only evidence the memory did not produce itself. A memory system that ranks what to keep by how often it is recalled can feed on its own output, and OpenClaw's history shows how. In April 2026 its dreaming promoted transport and session metadata into `MEMORY.md` as durable memory, the kind of entry ingestion now strips before it can become a candidate. Even the query-diversity floor had a hole until September 2026: the scheduler's own daily identifiers counted as distinct queries, so nightly ingestion alone could promote operational residue. The promotion threshold itself was calibrated on synthetic fixtures, durable facts at 0.750 to 0.756 against repeated filler at 0.489 to 0.549, so it separates the kinds of entry it was shown, not entries that helped. *Chapter 13* warns that an injected claim which survives into a trusted synthesis has been laundered into the system's own knowledge, and the defense is to name every signal that can nominate a memory and remove the ones the system produced itself.

Lesson 4: Let the assistant add to memory on its own, stage what it would delete, and name who reads the record. OpenClaw states why approval of learned changes left the default: the friction left corrections and successful work sitting as pending proposals instead of becoming skills. Hermes' opt-in staging showed the other side, a queue that grew to 74 pending items, 29 of them patches to one skill. What an after-the-fact record costs showed up in September 2026, when an unattended Hermes review fork, prompted to consolidate a nearly full memory file, deleted four standing permission rules from a user's memory. Nothing surfaced it, and the deletions were found the next day only by diffing the file against its last automatic commit. Hermes' fix is the asymmetry to copy: unattended additions to memory still land, and every unattended replace or remove in memory is staged for the operator. A record is the control only when someone reads it, so the destructive writes that reach it need a named reader and a way to roll them back.

Lesson 5: Gate what the assistant keeps on a replayed before-versus-after score. Without an outcome measure, the learning loop fails unseen in both directions. Under prompts that told it to file session-specific detail in reference files, one Hermes installation grew a skill whose `SKILL.md` reached 100,000 characters beside 443 per-session reference files, and in another a two-day denial storm left the loop running continuously while almost never landing a patch. A transient failure can harden into a rule as well. After one user moved to a new environment, the browser failed because a dependency was not yet installed, the review fork recorded that failure as a skill, and the agent refused its browser tool for weeks after the install was fixed. Hermes answered with a list of what the reviewer must not capture, environment failures and negative claims about a tool among them, which filters the write and still leaves its value unmeasured. Name the retention signal, since nothing in a memory system chooses it for you, and demote it to a filter that nominates candidates. Which candidates persist is a decision for an evaluation, and the book recommends one a team can build from traces it already records:

1. Hold out a set of past sessions the candidate was not learned from, weighted toward the obligations that have an oracle: bookings, authorized actions, and messages to the right recipient.
2. Replay each session twice, once with and once without the candidate memory or skill, holding the model and everything else fixed and pointing effectful tools at stubs so the replay books nothing and sends nothing.
3. Score both runs on those obligations, using the operator's corrections in the original session as labels where they exist, and promote the candidate only when the delta clears a margin fixed in advance; otherwise it stays staged or is archived.

The three steps are *Chapter 11*'s regression gate applied to a single memory or skill. OpenClaw's shadow trial existed for about eight weeks as a report-only runner and a test scenario, never

wired into promotion, before the project deleted it, and its personal-agent scenarios already encode the kind of obligation worth scoring. Obligations without an oracle stay ungated under this procedure, and until a gate like this exists, "learns over time" stays a hypothesis for either system.

Case study 6: Coding agents, what Claude Code and Codex build around the model

Anthropic's Claude Code and OpenAI's Codex are the harnesses two vendors built around their coding models, and reading them shows how much of a shipping agent is machinery around the model rather than the model itself.

The problem

Most of the systems in these cases were assembled out of models, tools, and glue. Coding agents are where the glue became the product. Two companies built one independently, at comparable scale, and left enough in the open to read: Anthropic documents the harness of its closed Claude Code, which has also been taken apart at source level, and OpenAI ships Codex as an Apache-2.0 repository. They are not a controlled comparison, since the teams saw each other's work and the products chase the same users. Where *Case study 5*'s two projects disagreed about which part of the problem was hard, these two vendors agree on which questions a harness must answer and disagree on the answers.

Those questions are this book's table of contents. A coding agent has to decide what the model may do without asking, what to carry in a context window that will not hold the repository, how to hand work to something else and get an answer back, how a user extends it, and what survives the session. Those are *Chapter 2*'s authority question, *Chapter 7*'s memory question, *Chapter 8*'s delegation question, *Chapter 9*'s protocol question, and *Chapter 13*'s persistence question.

The system both have to beat is a model in a bare loop: shell and file tools, and no permission policy, no compaction, and no delegation. Anthropic draws that line itself, between a client SDK in which the developer writes the tool loop and an Agent SDK that ships the tools, loop, and context management that power Claude Code [Anthropic, Claude Agent SDK]. What the agents' pull requests meet in review, test, and merge is *Case study 7*'s subject.

Architecture: What the two harnesses are built around

Chapter 1 put one number on the book's argument: in the version examined, about 1.6% of Claude Code's codebase was model-facing decision logic and the other 98.4% was operational harness [Liu et al., Dive into Claude Code]. The teardown reports that ratio as an estimate from community analysis of the extracted source. A ratio of lines says where code lives, not how

much each part matters, and the model's training lives in no repository at all. Codex, built by different people in a different language, shows the same kinds of mechanisms at the center of its repository. Codex is read at commit 94cbbdda of the `openai/codex` repository, dated 30 August 2026 [OpenAI, Codex Harness].

OpenCode, an MIT-licensed harness that neither model vendor built, is a third witness wherever it changes a reading, and Table D.2 stays with the two vendors. It runs models from many providers and keeps a separate base prompt for each model family, so the adjustment to any one model sits in the repository where a reader can see it. Where all three harnesses arrive at the same mechanism, the problem is a likelier cause than any one vendor's model; where OpenCode goes another way, the difference marks a choice. OpenCode is read at commit 82d4c890 of the `anomalyco/opencode` repository, dated 23 September 2026 [OpenCode, OpenCode].

Design patterns and trade-offs

Both products land on the same design choices without a framework between them. Each keeps the permission decision out of the prompt, and they disagree about where to put it instead. Each hands work to a subordinate context and takes back a result, the supervisor-worker form of *Chapter 6*'s Task Decomposition with the blast radius drawn at the context boundary. Each treats the context window as the binding resource and summarizes against a hard budget. Each reads its operating rules from files in the repository, which puts them under code review beside the code they govern.

The trade-off both accept is opacity in the live context. Compression that runs before every model call is compression the user cannot inspect as it happens, and summarization is a blocking inference stall that can retain different content across runs on identical inputs. A retained transcript still holds what the live prompt dropped; what is missing is a view of which parts the model was working from at each step.

Permission: Where the judgment and the rules sit

Anthropic reports that users approve 93% of permission prompts [Anthropic, Auto Mode]. A high approval rate alone does not show the prompts failing, since most requests may be legitimate, but it is the condition under which people stop reading them. What would settle it is how often dangerous requests get rejected and safe work interrupted, and whether users understood what they approved.

Claude Code answers by modeling the judgment. Its rules are evaluated deny first, then ask, then allow, and the first match decides, so no allow rule can carve an exception out of a deny [Anthropic, Claude Code Permissions]. Auto mode layers a second model over those rules in three tiers. Read-only tools and the user's allow rules pass first, file edits inside the project pass second because they are reviewable through version control, and only what remains reaches a

classifier: shell commands, web fetches, subagent spawns, and writes outside the project. On entering auto mode, Claude Code drops any allow rule known to grant arbitrary code execution, such as blanket shell access or a wildcarded script interpreter. A convenience rule written for manual mode would otherwise keep the most dangerous commands away from the judge. The classifier runs a single-token filter tuned to err toward blocking and, only on what that filter flags, a second pass with chain-of-thought reasoning over the same input. In the requests Claude Code sends to it, the classifier sees the user's messages, every tool call except read-only lookups such as file reads and searches, and the project's `CLAUDE.md`. The agent's own prose is stripped so it cannot argue its case, and every tool result is stripped so injected content does not reach the judge [Anthropic, Claude Code Permission Modes]. Anthropic tested the alternatives before settling there. Adding the agent's messages and tool results lowered performance on its test set of real overeager actions. A variant that saw tool calls alone missed the fewest dangerous actions but blocked work the user had plainly asked for, because it had lost the record of what the user wanted. A denial returns to the agent as a tool result with an instruction to find a safer path, and 3 consecutive denials or 20 in a session pause auto mode and return the decision to a human. A non-interactive run without a permission callback skips the blocked action and carries on when repeated denials reach the fallback threshold. The denial limit plays the part of *Chapter 8*'s circuit breaker, a threshold that trips, pointed at an agent that keeps pushing against a boundary.

Beneath all of it sits the OS-level sandbox: the permissions documentation says a rule matches only the command text Claude writes and is no security boundary, and it sends filesystem and network guarantees to the sandbox. The sandbox also lets the harness ask less often, and *Chapter 14* reports the resulting 84% cut in permission-prompt frequency [Anthropic, Containing Claude].

On 52 real overeager actions from internal sessions, Anthropic's deployed pipeline let 17% through, against a 0.4% false-positive rate on 10,000 ordinary tool calls and a 5.7% miss rate on 1,000 synthetic exfiltration attempts. In most of the misses the classifier saw the danger, the force-push or the deletion, and judged wrongly that something the user had said earlier covered it: "clean up the PR" is approval-shaped, and it does not name a force-push. Anthropic calls the result a substantial improvement for a user who would otherwise skip permissions and arguably a regression for one who reads every prompt with care. It rules out treating auto mode as a replacement for careful review on high-stakes infrastructure.

Codex answers by compiling the judgment into a program, and by separating two controls that a permission prompt folds together. Its documentation puts the split in one line: the sandbox mode sets what Codex can do technically, and the approval policy sets when it must stop and ask before crossing that line [OpenAI, Codex Approvals]. The policy program is the `execpolicy` crate, a Starlark rule engine whose rules match a command by its prefix and return one of three

decisions, allow, prompt, or forbidden, with a justification in prose that the approval prompt can show the user. A rule's `match` and `not_match` fields carry example invocations that are validated when the policy loads, so a rule that stops matching what its author meant fails at load time, and the permission policy has unit tests. When several rules match, the strictest decision wins, which is the same property Claude Code's deny-first order buys.

The design teaches most in how the sandbox, the approval policy, and the rules combine. Under the default on-request policy, a command no rule matches runs inside the sandbox without a prompt, unless the agent asks to escalate beyond the sandbox or a built-in heuristic flags the command as dangerous. Under the never policy the same prompt becomes a refusal, and the sandbox is the only protection left. An allow rule covering every segment of a command does more than skip the prompt: the command's first attempt can run outside the sandbox, so the rule is a grant of authority. Inside the writable workspace the sandbox still holds `.git`, `.agents`, and `.codex` read-only, so the agent cannot rewrite its own history or configuration. The workspace boundary itself has to come from the session: a 2025 advisory reported that the Codex CLI could treat a model-generated working directory as the sandbox's writable root, which release 0.39.0 fixed [OpenAI, Codex Sandbox Advisory]. Claude Code's classifier enforces the same boundary with a block rule against an agent modifying its own permission settings.

The two designs meet again at the reviewer. Codex can hand eligible approval requests to a reviewer agent [OpenAI, Codex Auto-review], and that reviewer fails closed on a timeout, a failed review session, or output it cannot parse. It makes the opposite choice about evidence: its transcript builder keeps both tool calls and tool results, because the reviewer often needs the exact path the agent queried and what came back. Tool evidence gets a separate budget so it cannot crowd out the human conversation. Anthropic's judge, without tool results, cannot tell whether a job identifier came from a query or was invented. Codex's judge can, and injected content reaches it. Both are defensible, and each is the answer to the second of the five questions, what evidence crosses the boundary into the component that decides.

These are two layers more than two philosophies, since Codex has a reviewer model and Claude Code's deny-first rules are a program too. A classifier generalizes to commands nobody enumerated and cannot tell you in advance what it will allow. A policy program tells you exactly what it will allow and is silent on everything its author did not think of. They also price the decision differently: the classifier spends a model call on each judgment it makes, and the policy program spends its author's time once. Both are instances of *Chapter 14*'s tool-call mediator, whose pattern leaves open the evidence question they answer differently.

The source-level teardown shows two cautions about the layered design. Defense in depth assumes its layers fail independently, and Claude Code's layers share cost and latency pressure, a sharper version of a claim *Chapter 14* makes in general terms. In the version the teardown

read, a command with more than 50 subcommands skipped the per-subcommand deny-rule checks and fell back to one generic approval prompt, because the per-subcommand parse froze the interface. The security researchers who reported the gap say v2.1.90 closed it. The second caution is one of timing. Hooks, Model Context Protocol (MCP) connections, and settings resolution all executed during project initialization, before the trust dialog and outside the deny-first pipeline, and the teardown traces two separately reported vulnerabilities, each patched within weeks, to that window. A reviewer hunting this class of bug reads the startup path, which *Hooks: the rules an instruction file cannot enforce* returns to below.

Context as the binding constraint

In v2.1.88, Claude Code ran up to five compaction layers before each model call, cheapest first, each aimed at a different kind of pressure: budget reduction, snip, microcompact, context collapse, and auto-compact [Liu et al., Dive into Claude Code]. Three of the five sat behind feature flags. Codex expresses the same commitment as separate compaction modules: a local path that summarizes on the client, and a remote path that summarizes on the model provider's servers. When the model that ran the earlier turns fails to compact them, a fallback retries on the current model.

What each keeps when it compacts is the more instructive half. Claude Code replaces the conversation with a structured summary and then rebuilds the rest from disk [Anthropic, Claude Code Context Window]. The system prompt, the project-root `CLAUDE.md` and unscoped rules, auto memory, and the plan all come back as written. Up to five of the most recently modified files are re-read, and each skill already invoked returns, capped at 5,000 tokens each and 25,000 across all of them. Rules scoped to paths, nested instruction files, and context that hooks added earlier load into the message history, so they are summarized away with everything else. The documentation's advice for a rule that must survive is to move it to the project-root file. Codex, on its local path, keeps the user's own most recent messages verbatim up to 20,000 tokens. Everything the agent said and every tool output gives way to a summary written, in its prompt's words, "for another LLM that will resume the task". It then discards any instruction messages carried in the old history and renders instructions and permission state afresh from the live session configuration. The two products split the window along the same seam, *Chapter 7*'s line between LLM-visible context and runtime state: what can be regenerated from configuration is regenerated, and only the conversation is entrusted to a summary. They differ in what they treat as irreplaceable, the user's words in Codex and the files on disk in Claude Code. Codex's framing also makes compaction *Chapter 8*'s session handoff inside a single session, a record written for a successor that was not there. The resumed model is told that another language model started the problem and wrote the summary, and the local path warns the user that long threads and repeated compactions make the model less accurate.

The Codex tree also treats changing prompt state as a synchronization problem. Instructions, permissions, tools, and environment state enter the model through typed `ContextualUserFragments`; after an initial snapshot, the runtime appends only the fragments whose world state changed. The repository's review rules state the contract: do not rewrite history, avoid context churn that breaks prefix caching, hard-cap every injected item at 10,000 tokens, and require additional manual review when one can exceed 1,000. Compaction manages old dialogue; typed diffs manage live state without repeatedly rebuilding the prompt prefix. OpenCode reaches both designs. Its compaction prompt asks for a summary "so another coding agent can continue the work", the same successor Codex writes for. Its newer session runtime keeps a typed snapshot of each context source and, when sources change, appends one system message at the next turn boundary, so the stored baseline stays the provider's cached prefix until compaction replaces it.

Compression reaches authority too. Auto mode re-reads a boundary the user states in conversation, such as "don't push", from the transcript on each check, so compaction can drop the message that set it [Anthropic, Claude Code Permission Modes]. Codex's local path keeps such a boundary verbatim only while it sits among the user's most recent 20,000 tokens. Anthropic's advice for a hard guarantee is a deny rule. Its memory documentation says the same of instruction files: `CLAUDE.md` is loaded as context, not enforced configuration, and an action that must be blocked needs a hook [Anthropic, Claude Code Memory].

Persistence is where the two products diverge again. Claude Code writes JSONL transcripts that support resume, fork, and rewind. A resumed session gets its permission mode back, but session approvals do not carry into a forked process, so authority survives a gap less completely than history does. Codex offers an opt-in staged memory pipeline with citation parsing, so a recalled fact can be traced to its origin, and telemetry on whether a retrieved memory was used. The pipeline runs when a root session starts and skips ephemeral and sub-agent sessions. Claude Code takes the other side of the same decision: a subagent definition can opt into a persistent memory directory of its own, scoped to the user, the project, or the local checkout [Anthropic, Claude Code Subagents]. Whether a delegate writes to long-term memory is an authority decision wearing a storage decision's clothes, a per-agent boundary *Chapter 13* argues a self-evolving system has to draw on purpose, and the two vendors drew it on opposite sides.

Undo: What the harness can take back

Auto mode's second tier lets in-project file edits through because, as Anthropic puts it, they are reviewable through version control, which makes the reach of undo the warrant for acting without asking, and the three harnesses draw that reach in three places. Claude Code's checkpoints rewind what its edit tools changed, and its documentation says they do not track files modified by shell commands [Anthropic, Claude Code Checkpointing]. In auto mode its

grant and its undo therefore coincide: edits pass and are checkpointed, and shell commands, which no checkpoint covers, go to the classifier. Codex once snapshotted the whole worktree into hidden git commits so a turn could be undone, and removed the feature in April 2026, leaving undo as a retired feature flag so that old configuration files still parse. What bounds an unprompted command there is containment: the sandbox's read-only `.git` protects the history the user would restore from. OpenCode keeps a shadow git repository apart from the user's own and snapshots the whole worktree before and after every model step, so a file that a shell command changed rolls back with the rest. Gitignored files, new files over 2 MB, and anything outside the worktree are not captured.

OpenCode also grants the most without asking. Its default rule allows every tool and asks only about paths outside the project, reads of `.env` files, and the same call repeated three times in a row. Its own specification says the shell is not sandboxed and runs with the user's filesystem, process, and network authority, and no model judges its calls; a plain non-interactive run rejects whatever would have prompted. Its client-server split was once an entry point of its own: the harness started an unauthenticated local HTTP server that let any local process, and through a permissive cross-origin policy any website, run shell commands with the user's privileges [OpenCode, Server Advisory]. Version 1.0.216 narrowed the origins the server accepts and 1.1.10 made it opt-in, and once enabled it still asks for no password unless the user sets one. The widest default grant of the three sits beside the widest undo, and neither reaches past the worktree.

Hooks: The rules an instruction file cannot enforce

Hooks are how a team turns a sentence in `CLAUDE.md` into a check the model cannot talk its way past. A hook is a command, an HTTP endpoint, an MCP tool, or a prompt or agent run on a model, which the harness invokes at a fixed point in the loop. Anthropic describes the property it buys as deterministic control: certain actions happen whether or not the model would have chosen them [Anthropic, Claude Code Hooks]. The events fall at three cadences, per session, per turn, and per tool call, and the ones that enforce policy are the ones that can block. `PreToolUse` can deny a tool call or rewrite its input before it runs. `Stop`, `SubagentStop`, and `TaskCompleted` can refuse to let an agent finish, so a team can hold an agent in its loop until the test suite passes, and `PreCompact` can refuse a compaction. A hook can only tighten the rules: a hook's `allow` does not override a `deny` or `ask` rule, and a hook's `ask` forces a prompt even in auto mode, where the classifier may still deny the call but cannot approve it silently.

The failure behavior is the detail a team gets wrong. For most events, only exit code 2 blocks on its own. Exit code 1, the conventional Unix failure, is a non-blocking error and the action proceeds; a hook whose script path is mistyped fails the same way, and a command hook that times out on `PreToolUse` does not block the call either. The documentation warns that a

mistyped path leaves the gate silently disabled. A policy hook therefore fails open unless it is written to fail closed, which is the opposite default from Codex's approval reviewer.

Codex adopted the same contract. Its hooks crate names its engine `ClaudeHooksEngine` and defines twelve events, eleven of them under Claude Code's names from `PreToolUse` and `PermissionRequest` to `SubagentStop` and `Stop`. It blocks on exit code 2 only when the hook also writes a reason to `stderr`; an exit 2 with nothing on `stderr` is recorded as a failed hook, and the call runs. A test comment records a place where it deliberately diverges from Claude's public hook documentation. One vendor's extension contract has become the other's, as `AGENTS.md` became a file both products read, which answers *Chapter 9's* protocol question by adoption. OpenCode did not adopt it. Its extensions are TypeScript plugins loaded into the harness's own process, and a plugin blocks a tool call by throwing an error before the call runs, so there is no exit code to get wrong. The failure default moves to loading instead: a plugin that fails to load is logged and skipped, and the session runs on without its guard.

A hook is also code the repository runs on the developer's machine. Claude Code's permissions documentation now states that hooks in a repository's settings files run when only a parent folder was trusted and in headless or SDK sessions, which do not show the trust dialog. A project's settings also take precedence over the user's, so they can switch hooks back on after the user disabled them. For a repository the developer did not write, the options include starting with `--bare` or passing `disableAllHooks` on the command line. By default, Codex runs a hook that is not centrally managed only after the user has trusted it. It records that trust against a hash of the hook's definition, so a hook edited after approval is marked modified and stops running until someone approves it again.

Public repositories suggest how little of this teams commit. A refereed study of 2,853 GitHub repositories configured for coding agents found that of the 1,297 configured for Claude Code, 94.3% committed a context file and 3.2% committed hooks [Galster et al., *Configuring Agentic Tools*]. Its authors conclude that harness engineering in open source "is therefore mostly context engineering". In the same sample, 493 repositories carried `AGENTS.md` and no tool-specific configuration at all, skills and subagents sat at a median of two per repository that used them, and 85.5% of skills shipped with no resources beyond the instructions. No repository in the sample committed the persistent memory Claude Code offers its subagents, so the per-agent memory boundary is one public repositories have not yet exercised. The study sees only what public repositories commit, so user-level settings, centrally managed policy, and private repositories are outside it; it bounds what teams publish, not what they run.

Where the team primitives live

Neither product was built as a multi-agent framework, and both grew coordination anyway, down to the same definition of a role. A Claude Code subagent is a Markdown file: a name, a

description the parent reads to decide when to delegate, optional limits on tools, model, and permission mode, and a body that replaces Claude Code's system prompt [Anthropic, Claude Code Subagents]. It starts in a fresh context window holding its own prompt, the parent's delegation message, the project's instruction files by default, and a git status snapshot, and none of the parent's conversation, auto memory, or files already read. It hands back a summary of its work, not the context it worked in. An `isolation: worktree` field gives it a temporary git worktree, and nesting stops at a configurable depth, three layers below the main conversation by default. A Codex role is the same thing in TOML: a file the `agent-roles` crate loads, with a name, a description the spawn tool shows the parent, and a layer of ordinary configuration applied to the child. The `agent-graph-store` crate records the parent-child topology as spawn edges marked open or closed. In both products a role is a prompt plus a configuration layer over the same agent loop.

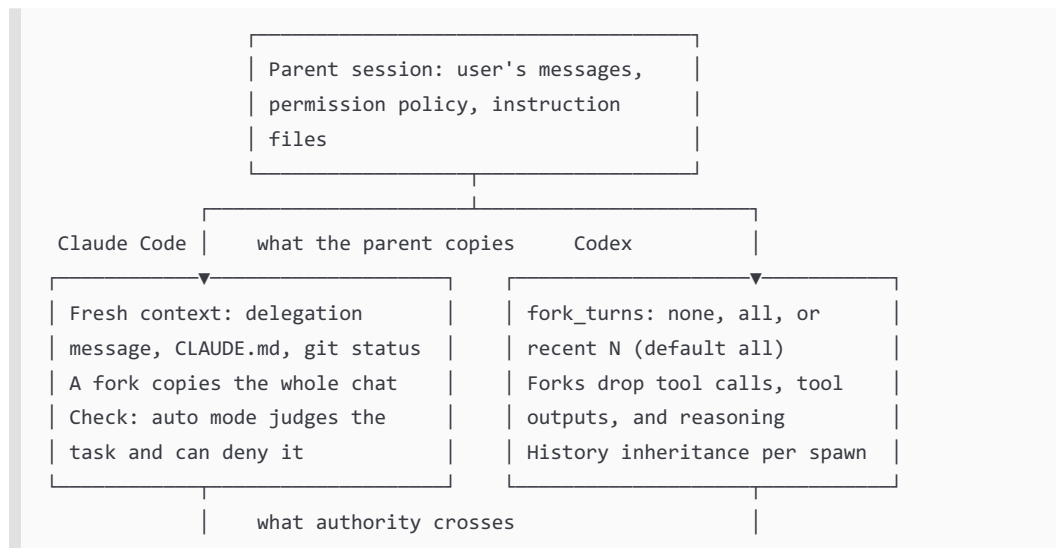
Delegation is then a choice of how much context to copy. Claude Code's fork is a subagent that inherits the whole conversation, the parent's system prompt, tools, and model, and shares its prompt cache, where any other subagent starts fresh. In the newer of Codex's two multi-agent tool versions, the spawn call takes `fork_turns` as `none`, `all`, or a number of recent turns, defaulting to `all`, and its description warns that a child given `none` may lack the context it needs. The older version takes a yes-or-no flag that defaults to the task message alone. Even a full Codex fork copies the conversation without the parent's work in progress: the forked history keeps system, developer, and user messages and the parent's final answers, and drops its tool calls, tool outputs, and reasoning. Both products ship *Chapter 7's context isolation* and *full-trace sharing* as a per-call switch. Codex's own delegation guidance adds a rule *Chapter 7's read many, write one* does not: when code-editing work is split, each delegate gets a disjoint write set.

Both then have to decide what authority crosses the handoff, and here they diverge. A spawned Codex agent inherits the parent's live approval policy, its approvals reviewer, its working directory, and its sandbox profile, and only then applies the role's layer. That layer may change instructions, model, and reasoning settings, and it may switch features and skills off but not on. The module's own comment says roles may reduce the child's capabilities but never replace the parent session's authority. The type that carries a role's overrides enforces *Chapter 14's* rule that delegation attenuates authority. Codex's internal delegates go further. At commit 94cbbdda Codex ran its code-review task and its approval reviewer as delegated sub-threads. Codex's delegate launcher rejected a delegate unless its approval policy was `never`, then carried the parent's selected environment and `execpolicy` into the child session. The approval reviewer has since moved into an extension of its own, still under `never`. Since `never` turns a prompt into a refusal, the child cannot negotiate broader authority, and work that does not fit the inherited sandbox and policy stays with the parent. The review delegate also loses web

search and the multi-agent tools, and the approval reviewer runs on a read-only intersection of the parent's permissions.

Claude Code lets a definition file move a child's authority in either direction, within a documented bound. A parent in auto, accept-edits, or bypass mode imposes that mode on the child whatever its file says, and in auto mode the classifier judges the child's calls against the parent's rules. Under any other parent mode the child takes the mode its file declares, short of bypass, so a subagent can accept edits without prompting under a parent that would have asked. Subagents shipped in plugins cannot set a permission mode, hooks, or MCP servers at all. Claude Code checks the handoff at both ends, and Anthropic states the reason plainly: inside a subagent, the orchestrator's instruction is the user message, so every task looks fully authorized [Anthropic, Auto Mode]. The classifier therefore judges the task at delegation, when it is still visibly the agent's choice and can be denied, and reviews the subagent's full action history before the report returns, warning the parent without discarding the work. Independently of the mode, the harness scans every subagent's final report for text imitating the harness's own messages, neutralizes it, and marks reports that mention permission settings; every tool call the report prompts still goes through the normal permission checks. Codex's approval reviewer acts on the same diagnosis [OpenAI, Guardian Delegation Review]. When it judges a sub-agent's action, it reads the root conversation's recent messages and counts only the human's own as authorization, so a grant another agent relays never counts. A later change, when enabled, also shows it the user messages of the thread that sent the task, so a restriction the delegation left out stays visible [OpenAI, Guardian Sender Context].

Figure D.2 sets the two handoffs side by side, from what the parent copies to what comes back and where each product checks it.



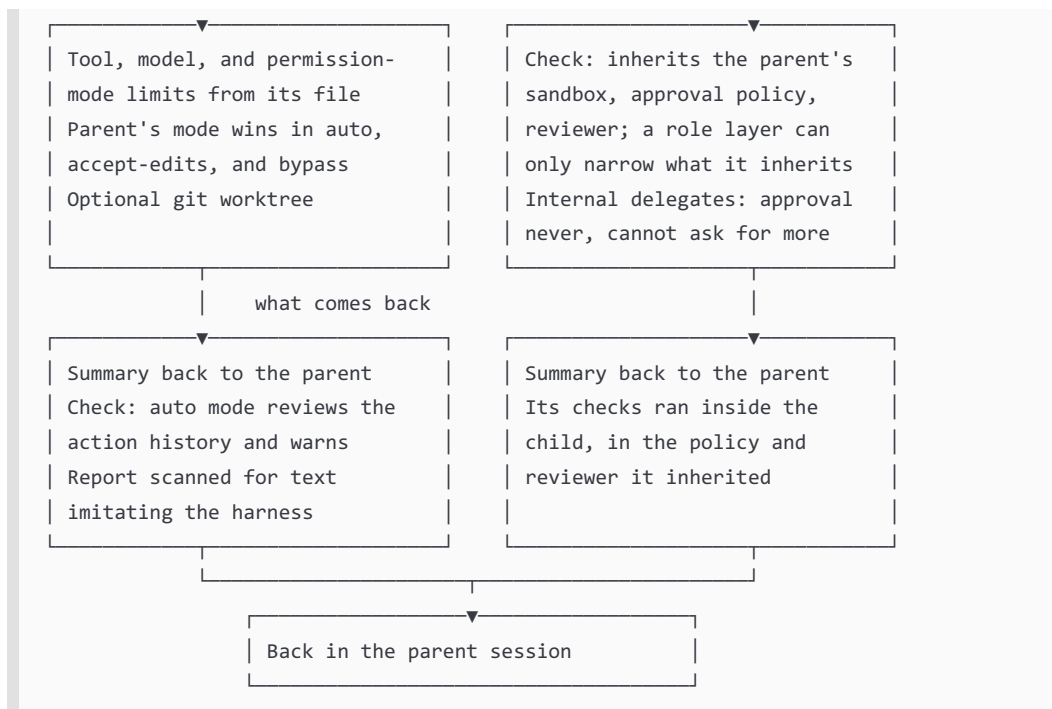


Figure D.2: Delegation in Claude Code and Codex. Both decide how much of the parent's context the child receives and return a summary. Claude Code checks the handoff at both ends and lets a subagent's file set its permission mode within a bound; Codex carries the parent's sandbox, approval policy, and reviewer into the child, lets a role only narrow them, and runs its internal delegates with no path to ask for more.

OpenCode's record shows what a restriction costs when it does not say whom it binds. Its plan mode denies edits through the plan agent's own rules, and in May 2026 a user showed that a subagent spawned from plan mode could still edit, because only the session's rules crossed the handoff. The same day's fix copied the parent agent's denies into every child. Within days, users who ran a restricted orchestrator over workers built to edit reported that the workers had lost the tools they existed to use. A compromise kept only plan mode's edit restriction, and a June change settled on the rule the source now states: "parent agent restrictions only govern that agent". A deny written on the session stays a ceiling for every child, plan mode now blocks the built-in general subagent by name, and a user-defined subagent runs on its own rules. One deny rule had been doing two jobs: a *ceiling* on everything a session does, and a *job description* that says what one agent is for and binds only that agent. The June rule makes the distinction structural: where a deny is written decides which kind it is.

Past a handful of delegated tasks, Claude Code moves the plan out of the model. Its dynamic workflows are scripts that hold the loop, the branching, and the intermediate results, with a documented cap of 1,000 agents per run [Anthropic, Claude Code Workflows]. The documentation separates subagents, agent teams, and workflows by who decides what runs next, not by how many agents run, which is the book's position since *Chapter 1*: agent count is not the axis. Two products nobody would file under "multi-agent systems" grew roles, a parent-child graph, delegation, and worktree isolation, which suggests that the problems producing those primitives are problems of authority, context, and concurrency, whatever the product is called.

Table D.2 sets the two designs side by side.

Question	Claude Code	Codex
Where the permission decision lives	Deny, then ask, then allow rules under a permission mode, plus an auto-mode classifier, itself a model, that sees user messages, tool calls, and CLAUDE.md in Claude Code's own requests, never tool results	execpolicy Starlark rules (allow / prompt / forbidden) with load-time example tests, a sandbox mode, and an approval policy, plus a reviewer agent that sees tool calls and results
How the human is protected from fatigue	Fewer decisions: OS-level sandboxing cut prompt frequency by a reported 84%; auto mode returns the decision to a human after 3 consecutive or 20 total denials	Fewer surprises: each rule carries a justification the prompt can display; commands inside the sandbox run without a prompt
How context pressure is handled	Layered compaction, cheapest layer first; after a summary, instruction files, memory, recent files, and invoked skills reload from disk	Local and remote compaction; the local path keeps the user's recent messages verbatim and re-renders instructions and permissions from configuration

Question	Claude Code	Codex
What survives a session	JSONL transcripts with resume, fork, and rewind; permission mode restored on resume, session approvals not carried into a fork; optional persistent memory per subagent	Opt-in staged memory pipeline with citation parsing and read-usage telemetry; skipped for sub-agent and ephemeral sessions
How work is delegated	Markdown-defined subagents with fresh context and summary returns, or forks with the whole conversation; optional worktree; in auto mode, classifier checks at delegation and return	TOML roles that can only narrow the parent's authority; history inheritance set per spawn; internal delegates inherit the parent's policy and cannot request approval
How a user extends it	MCP, plugins, skills, hooks that block on exit code 2, CLAUDE.md or AGENTS.md	MCP, plugins, skills, connectors, AGENTS.md, and hooks on Claude Code's contract, run only once trusted by hash
What the vendor treats as reusable	The harness, exposed through an SDK	The harness, released under Apache-2.0

Table D.2: Claude Code and Codex, question by question

Results and lessons learned

No number in this case says which of the two designs works better. Nobody has run the two systems against each other under a controlled protocol, a vendor's benchmark claim about its own product is marketing, and Anthropic's classifier evaluation measures one layer against its own two baselines. Every structural claim is also bound to the snapshots named above and to documentation read in September 2026, and the products ship weekly. What the case offers is architecture read from primary sources and one published measurement of a permission layer. OpenCode agrees with both vendors on compaction into a structured summary, instruction files read as context, and delegation that hands back only the child's result. It differs on the

permission and extension machinery, with no OS sandbox, no model judge, and in-process plugins in place of the hook contract. One more harness is a single data point, and its builders could read Codex's source and Claude Code's documentation, so agreement may be imitation as easily as necessity. Still, it suggests that context and delegation are what the work forces on any builder, and that layered permission machinery is what a vendor chooses to build. The lessons below are the ones that survive the version numbers.

Lesson 1: Compaction can quietly drop a limit, so enforce hard limits with machinery the harness reloads, not with words in the context. Claude Code reloads its project-root rules and summarizes its path-scoped ones away until a matching file is read again, and Codex keeps a boundary the user typed only while it sits inside the last 20,000 tokens. Both products regenerate what configuration holds and summarize only the conversation, so hard limits belong where they are regenerated: deny rules, hooks, policy programs, and the OS sandbox. Give every rule examples like Codex's `match` and `not_match`, so continuous integration (CI) fails a rule that has drifted.

Lesson 2: Decide what the judge may see, and choose its error on the actions that matter. Withholding tool results keeps injected content from the judge and costs it provenance, and the two vendors chose opposite sides of that trade. The error has to be chosen as well as the evidence: Anthropic's fast filter alone blocked 8.5% of ordinary calls and missed 6.6% of real overeager actions, and the reasoning pass took those to 0.4% and 17%. The deployed pipeline is an operating point, not a strict improvement, and most of its misses are consent failures, where the judge saw the danger and read an earlier user message as approval for it. A team building its own permission layer should measure its answer on its own overeager actions, since a false-positive rate on ordinary traffic says nothing about the actions that matter.

Lesson 3: A deny rule is a ceiling or a job description, so say which, and check it at the handoff. Write a deny on the session when it must bind every child and on the agent when it describes only that agent's job; OpenCode reached that rule only after each of its reversals broke one group of users. A delegated instruction reaches the child as if the user had given it, so the check belongs at the handoff. Codex attenuates by construction, and Claude Code lets a definition file set the child's mode within a bound and puts a judge at both ends; a team adopting either should know which of the two it is relying on.

Lesson 4: Grant without asking only what the harness can take back. A team adopting any of these harnesses should write down where its undo line runs and check that every unprompted grant falls inside it. The check is cheap: have the agent change a file through a shell command, rewind, and see whether the file comes back. Whatever no snapshot reaches, a push, a deploy, a sent message, or a write outside the project, needs a deny rule, a sandbox, or a person before it runs.

Lesson 5: Decide what a guard does when it breaks, because in all three harnesses a broken hook or plugin lets the work through. A Claude Code hook that exits 1 or times out, a Codex hook that exits 2 without a reason, and an OpenCode plugin that fails to load all let the work proceed, the failure *Case study 1* in *Chapter 16* met in CAIPE's policy solver. The same vendors fail closed where they thought about it, in Codex's approval reviewer and its hash-checked hook trust. *Chapter 15* asks for a declared policy on what a component on the call path does when it is unavailable, and for a hook that policy was declared by whoever wrote the harness. Write a `PreToolUse` guard that exits 1 and watch the call run, then make the policy hooks fail closed and test them the way Codex tests its rules. Hooks are already rare, committed in about one Claude Code repository in thirty, and a guard nobody has seen fire may be doing nothing at all.

Case study 7: Autonomous software engineering, multi-agent code generation and review

This case follows an agent's work through a team's software lifecycle, from a teaching composite of role-separated agents to the refereed field record of agent-authored pull requests.

The problem

Case study 6 reads inside the harness around a coding agent; this case asks what happens when that agent's output enters a team's software lifecycle. A change there passes through requirements, decomposition, implementation, review, test, and merge, and a team then owns the result for years.

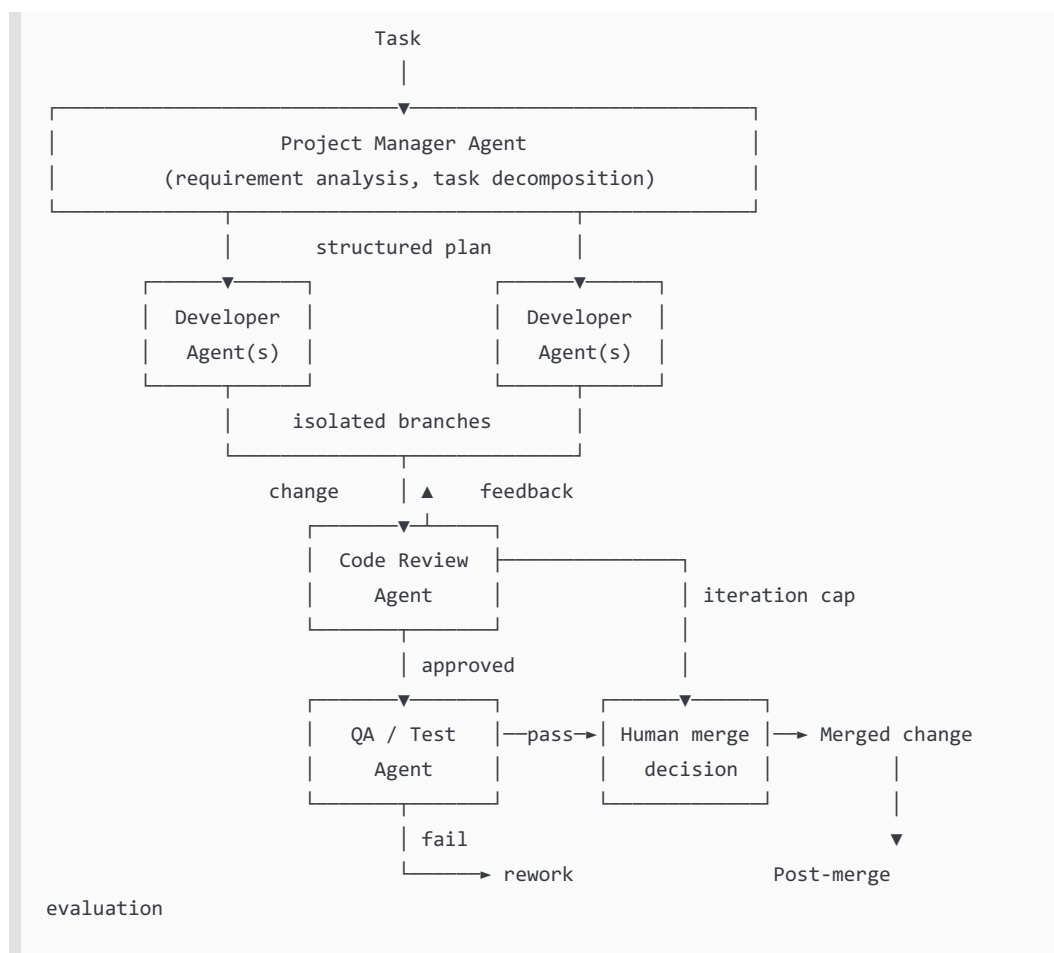
The baseline to beat is the team most organizations already run: engineers with ordinary code review and CI, each working with a single coding agent. Splitting the lifecycle across agents earns its coordination cost only where it beats that team on the same tasks, tools, budget, and human gate.

Anthropic's 2026 Agentic Coding Trends Report cites internal research finding that its own engineers' productivity gains show up mainly as more output rather than faster tasks [Anthropic, Coding Trends]. The SWE-bench benchmark tests agents on real GitHub issues from popular open-source repositories [Jimenez et al., SWE-bench], and on its public leaderboard in mid-2025 the leading entries resolved roughly 70 to 75% of the Verified subset. Those scores are now a closed historical record: in February 2026 the benchmark's own steward retired SWE-bench Verified as a frontier measure, reporting that 59.4% of the hard tasks it audited had flawed test designs and that every frontier model it tested showed signs of training on benchmark solutions [OpenAI, SWE-bench Retired].

Even a sound benchmark measures one stage, handing the agent an isolated issue with a clear acceptance test. A team hands it ambiguous requirements, a large unfamiliar codebase, conventions, dependencies, and code people will maintain long after the task closes, and most of that work sits before and after the code is written.

Architecture

The architecture is a teaching composite that follows the MetaGPT insight [Hong et al., MetaGPT], the standard operating procedure (SOP) variant of Task Decomposition *Chapter 6* introduced: specialized roles collaborating through structured artifacts rather than free-form chat. It is the shape a team would propose, and the field record in *What the pull-request record shows* pushes back on it. Its four roles are lifecycle responsibilities; whether they run as stages in one runtime or as separate agents is an implementation choice driven by context, authority, concurrency, and ownership. Figure D.3 shows the artifacts that pass between them:



pipeline)

(outside agent

Figure D.3: The code generation and review pipeline. The project manager agent turns the task into a structured plan, developer agents implement its sub-tasks on isolated branches, and the code review agent returns feedback until it approves the change or escalates at its iteration cap. QA failure returns the change for rework; QA success reaches an explicit human merge decision. Post-merge evaluation sits outside the agent pipeline and measures the repository the team inherits.

The project manager agent reads the directory tree, dependency graph, and recent commit history through tool calls, then writes a plan that gives each sub-task the files to modify, the expected behavior change, the acceptance criteria, and its dependencies. Those fields let a developer start without reinterpreting the request.

The developer agents implement the sub-tasks, concurrently where they are independent and in sequence where one depends on another. Each has file read/write tools, can run code, and works on its own feature branch, running local tests there before it submits.

The code review agent reads every change before merge for correctness, security, team conventions, and test coverage, the debate-and-reflection pattern from *Chapter 6*. It returns structured feedback the developer must address until the reviewer approves, and escalates a change still open at its iteration cap to the human who owns the merge.

The QA agent analyzes the diff, generates unit and integration tests for the behavior it changes, runs them, and reports the results as the last automated gate before the human merge decision.

Design patterns and context

Around the roles, the workflow is a sequential cascade from decomposition through review to testing (*Chapter 6*). Its context is layered the way *Chapter 7* describes: repository structure loaded once and shared, the files the plan names loaded with the unmodified code summarized, agent dialogue compacted as feedback resolves, and an optional episodic store of lessons from earlier tasks. Practitioners report that the store helps, with no measurement cited here, so it earns its place only when a before-and-after comparison on the team's own review outcomes shows the lessons helping.

The plan's acceptance criteria do the job *Chapter 6* gives a sprint contract, where generator and evaluator agree on what "done" means before generation starts. Written before any code exists, they give the reviewer and the QA agent the target the developer was handed, so neither infers the intended behavior from the change it checks.

What the pull-request record shows

The lifecycle the composite describes now leaves a public record. Coding agents leave recognizable marks on the pull requests they open, such as a bot author, a branch prefix, or a co-author line, so their work is auditable in ordinary repositories. The AIDev dataset collects it: 932,791 agent-authored pull requests from five agents across 116,211 repositories, with a curated subset of 33,596 drawn from repositories above 100 stars [Li et al., AIDev]. Several refereed studies read that record stage by stage, showing where agentic changes stall, how much review a merge reflects, and what their tests check.

Most rejected agentic requests did not fail in the code. A study of the 33,596 curated requests found 71.48% merged [Ehsani et al., Agents Fail]. Across agents, documentation, CI, and build changes merged most often (84%, 79%, and 74%), and bug fixes and performance changes least often (64% and 55%). Of the 600 rejected requests it sampled, the authors could still open 562, and the largest group, 228, had closed with no meaningful human engagement. Next came 142 duplicates of work already open in another request and 24 features the maintainers did not want; 99 failed the project's own CI or tests, and 34 were incorrect or incomplete. Each additional failed CI check lowered the odds of a merge by about 15%, and maintainer feedback kept asking for small changes limited to one coherent purpose. A manual reading of 353 rejected requests from repositories above 500 stars points the same way [Peralta et al., Merged or Rejected]. It found 35.7% reflected a clear agent failure, 31.2% closed for workflow reasons such as duplicates, superseded changes, or inactivity, and 33.1% left no observable rationale.

The review agent is where the record pushes back hardest. Across 3,109 pull requests drawn from AIDev, requests reviewed only by code-review agents merged at 45.20% (127 of 281) [Chowdhury et al., Code Review Agents]. Those agents include rule-based scanners, such as GitHub's code-scanning bot and Semgrep, as well as model reviewers. Requests whose human-only review also stayed in the neutral commented state merged at 68.37% (804 of 1,176). No agent in that sample approved or requested changes on its own: every request reviewed only by agents stayed in the neutral commented state, so the agents advised and someone else decided. The same study found low comment signal alongside the merge gap: 12 of the 13 review agents on the closed agent-only-reviewed requests averaged under a 60% signal ratio, the share of their comments that carried actionable feedback, and 60.2% of those requests fell in the 0–30% band. That is an association, and it does not isolate comment quality as the cause of the lower merge rate. Its authors draw the design consequence directly: configure review agents for narrow checks such as security vulnerabilities or style violations, and keep human approval before merge.

The merge record also shows less review than the diagram implies. The same manual reading inspected 364 merged requests and found 288 of them, 79.1%, with no visible loop of reviewer feedback and agent revision and no reviewer commit, with 15.4% requiring explicit reviewer

involvement. Where reviewers did step in, the fixes mostly concerned missing tests, CI configuration, or repository conventions, not core functionality. The per-agent split is wide: Copilot carried feedback or a reviewer commit on 29 of 49 merged requests, Devin on 25 of 107, and Codex on 1 of 167. Claude Code contributed 8 merged requests to that sample, too few to read anything from. The authors tie the split to the repositories each agent works in, where stricter CI gating and review norms went with more intervention, so a merge rate measures the review environment as much as the agent. The record also shows who holds the gate: 77.51% of merged agentic requests in the curated AIDev subset were merged by the same account that opened them, against 57.63% of human-authored ones [Yoshioka et al., Meaningful Pull Requests]. Merged agent-authored requests are also smaller and more localized than merged human ones, and their descriptions match their diffs at least as closely as human descriptions do [Ogenrwot & Businge, Modify Code]. Visible review activity is an imperfect measure, since a reviewer can read a diff and approve it without leaving a trace, and merged and rejected samples carry their own selection effects. The record is also public GitHub, mostly repositories above 100 or 500 stars as the agents behaved when AIDev captured them, and its authors caution that proprietary codebases with other review norms may differ. *Chapter 11* argues that task completion is necessary and nowhere near sufficient. A merge label is task completion with the reviewer's work hidden inside it, so merge rate alone tells a team little about what the gate checked.

One maintainer team's ten-month account of its own repository shows where the work binds once the agent is inside the lifecycle [Microsoft, Ten Months in dotnet/runtime]. Of the 878 requests GitHub's Copilot coding agent opened in the .NET runtime repository, the team merged 38.1% of the decided ones created before its first setup changes, against 69% of those created after. The changes were all to the agent's environment: firewall rules that let it reach the package feeds its build needed, a repository instruction file, and setup steps that got its builds past their timeouts. Review concentrated as well: two maintainers wrote 36% of all human feedback on the agent's requests.

What the test stage checks

The QA agent's gate is worth what its tests execute and what their assertions pin down, and the record mostly measures the first of those. A study of 4,882 agentic pull requests in Java and Python classified their test changes, then rebuilt the merged ones it could instrument and ran each repository's suite with and without the agent's test edits [Dipongkor et al., Test Coverage]. What it measured is *diff coverage*, the share of the lines a change adds that some test executes. Of the 4,387 requests that changed code under test, 2,211 (50.4%) changed no test at all. The existing suites made a thin net: in 1,079 of 1,664 merged Python requests, no existing test executed a single line the agent had changed. The agents' own tests raised coverage of their changed lines in only 136 of 605 Python requests that carried tests and 23 of 64 in Java. Most of the Python requests without a gain had added new tests that still missed the agent's

own lines. Error handling was the weakest spot in both languages, with more than four in five added lines inside exception-handling blocks left unexecuted. A lexical study counts what agent tests assert: no assertion pattern in 71% of the test files agents created, and a strong one, such as a value comparison, in only 27% [Banik et al., All Smoke].

That measurement splits in two the failure practitioners call test-suite erosion. The measured half is a test that never executes the change, which a diff-coverage check catches mechanically. The unmeasured half is a test that executes the change and asserts whatever it happens to do, so it passes when the implementation is wrong in a self-consistent way. None of the studies here measures it for agent-written tests, so it stays a hypothesis.

Meta's TestGen-LLM, an industrial system reported outside this composite, shows how that second failure follows from the design [Alshahwan et al., TestGen-LLM]. It extends existing Android test classes with model-written cases and keeps a candidate only if it builds, passes on each of five repeated runs, and raises coverage, three deterministic filters that need no model judgment. Its designers discard every failing candidate, because without an automated oracle they cannot tell a test that found a bug from one with a wrong assertion, so every test that survives protects the behavior the code already has. Across three internal test-a-thons it improved 196 of the 1,979 test classes it was applied to, and engineers accepted 73% of its proposals. One of its tests executed a previously uncovered method and cleared every filter, and the reviewing engineer rejected it because it contained no assertion. *Chapter 15* separates tests, which prove declared invariants, from evaluations, which estimate behavior. A model-written test declares whatever the code did when the test was written. *Chapter 6* treats passing tests as the clean external signal that lets a generator grade against the world, and the coverage study bounds that signal to the lines the tests run.

Results and lessons learned

Practitioners quote time-to-merge, review-cycle, and test-pass ranges for systems like this one that no publication this book can trace, so the composite repeats none of them. The lessons assume a harness configured for role separation, a real cost a team pays before any of them applies, and each is a hypothesis to test against the baseline team.

Lesson 1: Evaluate what gets worked on, and how big, before how it is built. A wrongly chosen or wrongly scoped task commits every later stage to the wrong work, and downstream agents cannot compensate. A developer agent handed a sub-task that spans two modules cannot narrow it, and a reviewer cannot tell an incomplete implementation from a wrongly scoped one. Among the rejected requests the failure study could read, duplicates of work already open outnumbered incorrect or incomplete code four to one, 142 against 34. Size compounds the problem, since conflicts with the base branch rise from about 10% of the smallest agentic requests to about a third of medium-sized ones [Ogenrwot & Businge, AgenticFlict]. So the project manager should read open pull requests and issues along with the

commit history before it plans, and size each sub-task to the one coherent change maintainers kept asking for.

Lesson 2: A second reader counts only if it brings evidence the author did not, and the merge stays with a human. Code that is coherent but wrong passes any reviewer that shares its author's assumptions, so a review stage catches what the developer missed only when it brings the plan's contract, fresh tests, static analysis, a different model or rubric, or a human's judgment. The idea module in *Chapter 16's Case study 2* shows why a critic that shares the generator's model and context is not independent. In the review study, requests where agent reviewers outnumbered human ones merged at 63.25% against 45.20% for agent-only review, and no mix that added agents beat human-only review on merge rate, an association and not a measure of what the review caught. Whether a review agent earns its place is *Chapter 11's* counterfactual test: run the pipeline with and without it on the same changes, models, and budget, and report the change in accepted outcomes and the change in cost as two numbers.

Lesson 3: Branch isolation moves conflict to the merge; it does not remove it. A branch prevents simultaneous overwrite of a shared worktree, but 27.67% of the open or unmerged agentic requests the AgenticFlict study could simulate conflicted textually with their base, from about 15% for Copilot to about 32% for Codex. Textual conflict is the part a merge tool can see. Semantic conflicts, stale assumptions, incompatible schema changes, and merge-order effects are not in that count; *Chapter 8's* renamed-function example is the kind a branch hides until integration. Dependency-aware scheduling, integration tests, and a merge queue therefore remain load-bearing. The worst version is architectural drift, code that is locally correct and globally incoherent. The Claude Code teardown predicts it from inside one harness: a subagent returns a summary instead of its working context, so parallel workers can independently re-implement what the repository already has [Liu et al., Dive into Claude Code]. A refactor, an interface redesign, or a cascading dependency upgrade yields no independent sub-tasks, so reserve parallel workers for well-scoped, well-specified work.

Lesson 4: Gate the test stage on what the tests execute and assert. Test presence is a weak signal: among Java requests that the existing tests did not already fully cover and that gained no coverage, agents deleted more tests than they added, 82 against 31. A QA gate that controls the merge needs a diff-coverage floor on the changed lines, error paths included, and assertions derived from the plan's acceptance criteria. TestGen-LLM's funnel shows why such filters are needed, since 57% of its generated test cases passed reliably and only 25% added coverage, and its rejected assertion-free test shows that the assertions still need a reader.

Lesson 5: Measure the repository the team inherits, because the speed-up fades and the complexity stays. A difference-in-differences study compared 806 open-source repositories that adopted Cursor's agentic editor with 1,380 matched repositories that did not [He et al., Cursor Velocity]. Lines added rose 281% in the first month and 48% in the second, and then the

gain was gone, while static-analysis warnings rose about 30% and code complexity about 41% and stayed up on the study's main estimator. In the authors' panel model the accumulated complexity then drags on later development velocity, so the cost of the early speed-up is paid after the gain has gone. A merge rate cannot see any of this. Instrument the repository over months, alongside the merge, and set test-coverage requirements that scale with the lines a change adds, which is the authors' own recommendation.

Case study 8: Financial services, governed risk analysis and compliance

The first seven cases began with systems and extracted answers to the five questions from them. This last case runs the questions forward: it begins with a consequential workflow and derives the smallest architecture that satisfies its constraints, taking the baseline first, then authority, evidence, and the evaluation that controls release. It is an illustrative case study: it describes no named deployment and claims no measured outcome. The setting is real, though: where a design choice rests on rule text or supervisory guidance, the case reads that source for what it says and no further. None of this is legal advice, and which rules bind a particular firm depends on its charter, its business, and its jurisdiction.

An asset manager monitors portfolios against regulations, client investment policy statements (IPSS), and internal limits. A fictional issuer, Acme Corp, has its 2028 bond downgraded from investment grade to high yield. The change may push a client's high-yield exposure above a limit, but a useful finding requires more than noticing the rating event. The system must calculate exposure from the positions actually held and establish which version of the client's policy was in force. It must also distinguish external requirements from stricter internal policy, explain ambiguity, and put a qualified person in control of the response.

Start with the system that must be beaten

The baseline is not a person scanning an end-of-day report. It is an intraday rules engine, retrieval over a governed policy corpus, and a qualified compliance reviewer, and it already does more than a first design tends to credit. The engine holds the client's high-yield limit as a coded rule whose threshold comes from the IPS. When the rating feed reclassifies Acme's bond, the engine recomputes exposure, finds it above the limit, and raises an alert that carries the rule, its inputs, and the numbers. Detection is finished before any model is involved.

What remains is the disposition, and that is where the reviewer's time goes. A limit exceeded because a rating changed, with no trade behind it, is what practitioners call a passive breach, and the rules that govern it often differ from those for a breach the manager caused. The EU's Undertakings for Collective Investment in Transferable Securities (UCITS) Directive shows the shape: when a fund exceeds its issuer-concentration limits for reasons beyond its control, it

must make remedying the situation a priority objective for its sales, taking due account of its unitholders' interests [EU, UCITS Directive]. The rule sets a priority and a standard of care, with no deadline and no forced sale, so what happens next is a judgment the rule leaves to the manager. A client's IPS may add terms of its own, such as a period during which a downgraded bond may still be held, and the firm's internal policy may be stricter than either. So the reviewer opens the IPS version in force on the event date with its amendments, finds the clauses that govern a downgrade, checks the internal escalation rule, and writes a disposition with the evidence behind it. That assembly and interpretation is the step an agentic addition can plausibly improve.

Every agentic addition must improve a declared, decision-relevant measure on the same portfolios without increasing independently ascertained misses or weakening the audit record. Because the engine already detects the exceedance, an addition could raise the miss rate only by standing between the engine and the reviewer with the power to delay, reorder, or drop the alert. The design gives it none of those powers. Nasdaq Verafin plans to grant one of them: its announced auto-dispositioning will let select agents in its workforce close false-positive alerts themselves and surface only the rest for human review [Nasdaq Verafin, Agentic Workforce]. For its sanctions agent the vendor reports workload reductions of up to 90% from initial customer results, with no figure for alerts closed wrongly.

So the first design question is where model judgment adds value, and arithmetic is not one of those places. Position lookup, security classification, exposure aggregation, and threshold comparison remain deterministic and independently reproducible. Classification stays in code even where its rule is written in prose. An IPS may say which agency's rating governs when the agencies disagree; that clause is read once, encoded, reviewed, and versioned, and no model reinterprets it on every event. Model judgment may help interpret prose, reconcile overlapping policy layers, explain why the result matters, or identify what evidence is missing. That split applies *Chapter 4's* tool discipline, *Chapter 7's* context discipline, and *Chapter 11's* paired comparison against a baseline before it applies any topology from *Chapters 6 and 8*.

The split is also a line between governance regimes, which makes it the first answer to where authority sits. The US banking agencies' revised model risk guidance defines a model as a complex quantitative method that applies statistical, economic, or financial theories to turn input data into estimates [Federal Reserve, SR 26-2]. It excludes simple arithmetic, and deterministic rule-based processes with no such theory behind them. The same guidance places generative and agentic AI outside its scope as novel and rapidly evolving, and leaves the controls for them to each organization's own risk management and governance practices. That revision dates from April 2026, so this part of the setting is the first to re-read before a design decision rests on it. The guidance is written for banking organizations, and an asset manager outside a bank is not its addressee; the case borrows its definitions because they draw the line this design draws. The arithmetic that stays in the rules engine stays under controls the firm

and its auditors already run; the interpretive step is the part whose controls the firm has to design. Broker-dealer supervision makes the complementary point from the other side: the Financial Industry Regulatory Authority (FINRA) reminds its members that its rules apply when they use generative AI just as they apply to any other technology, and says the reminder creates no new requirements [FINRA, Notice 24-09]. Adding a model changes how an answer is produced, not who answers for it.

Follow one event across explicit boundaries

Figure D.4 follows the downgrade from event to decision and marks the two lines the design is built around: where model judgment is allowed, and where the authority to decide and to act sits.

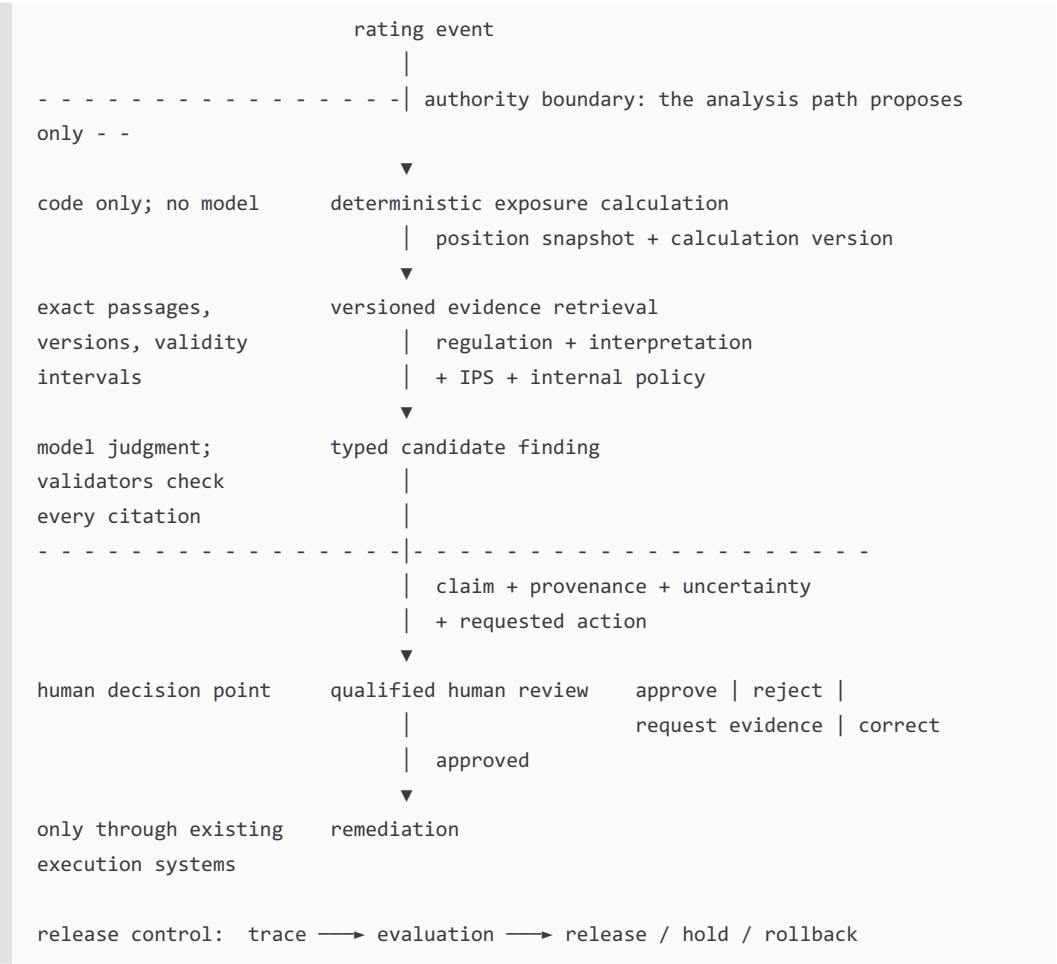


Figure D.4: The illustrative compliance path from rating event to decision. Code calculates the exposure and retrieval returns exact, versioned passages; model judgment is confined to drafting the typed candidate finding, which validators check; a qualified reviewer decides; and remediation runs only through existing execution systems. The trace feeds the evaluation that releases, holds, or rolls back a change.

The event handler first resolves the downgraded security to holdings and passes a timestamped position snapshot to deterministic calculation code. If the portfolio held 12% high yield before the downgrade and the reclassified bond adds 4 percentage points, the calculation reports 16% and records the inputs, rounding rules, and calculation version. A model does not redo this arithmetic in prose.

The next boundary gathers the evidence needed to interpret that number, and what crosses it is this workflow's answer to the second question. The corpus keeps its collections separate:

1. **Primary requirements:** applicable regulations and formally issued rules, versioned by effective date and jurisdiction.
2. **Interpretations:** guidance, formal clarifications, and approved legal or compliance analysis, linked to the primary text they interpret.
3. **Client instructions:** the signed IPS and approved amendments for this account, with validity intervals.
4. **Internal policy:** the firm's own limits and escalation rules, which may be stricter than an external minimum.

Separation preserves provenance. A model summary does not become a source, and an internal preference is not presented as law. Retrieval returns exact passages, document identifiers, versions, and validity intervals. When two layers conflict, the output records the conflict without silently blending them.

The version history is itself a record-keeping duty in parts of this setting. An investment adviser registered with the US Securities and Exchange Commission must keep a copy of the compliance policies and procedures that are in effect, or were in effect at any time in the past five years [SEC, Rule 204-2]. So "which internal policy governed on the event date" has a recorded answer, and the retrieval layer can be tested against it. *Chapter 7* argues that a shared store becomes debuggable, and in regulated settings compliant, only when every change records who wrote it, when, and what the prior value was.

Those responsibilities do not automatically require separate agents. Retrieval deserves an independent service or agent only if its data ownership, access policy, evaluation, or release lifecycle differs from the rest of the workflow. A second reviewer role is useful only if it brings independent evidence or judgment; calling the same model twice does not create independence. Each coordination boundary has to buy something specific: concurrency,

evidence access, authority separation, or lifecycle independence. Otherwise, keep the work in one tool-using loop or in ordinary workflow code.

Make the handoff a contract, not a paragraph

The interpretive stage produces a `CandidateFinding`, not a finished compliance decision. Its fields are *Chapter 5*'s seven handoff-contract elements, and Table D.3 fills them in for this domain.

Field	What it carries here
claim	Portfolio P may exceed its high-yield limit after event E
provenance	Source passages, document versions, position snapshot, rating event, and calculation version
uncertainty	What is known, disputed, missing, or out of distribution
requested_action	Review classification and choose a disposition
authority_boundary	Analysis only; no trading or policy-edit authority
deadline	Review-by time derived from the applicable procedure
expected_response	One of approve, reject, request_evidence, or correct

Table D.3: The fields of a `CandidateFinding`

The contract does three jobs. It prevents a persuasive narrative from hiding missing evidence. It lets downstream systems reject a malformed or incomplete handoff before anyone acts on it. It also makes replay possible: a later audit can reconstruct the claim from the referenced source versions without trusting whatever text happened to be in a prompt. *Chapter 11* draws the same line for evaluation: a trace that keeps the documents and tool responses it consumed can be rerun, and one that keeps only the model's text can be read but not rerun.

The model may draft the plain-language explanation: Acme's downgrade moved the portfolio's classified high-yield exposure from 12% to 16%, above the 15% cap in the client's then-current IPS. Because no trade caused the exceedance, the IPS's downgrade clause and the firm's cure procedure govern what happens next. That reading of prose is where the model earns its place, and it is also where it is most likely to be wrong. Deterministic validators still check that every cited document exists, was effective at the event time, and is authorized for

that portfolio. Whether a passage supports the field that cites it is a reading, not a lookup, so it belongs to the reviewer or to a judge whose error rate has been measured.

Put authority at the eventual effect

The analysis path can read positions and policies, calculate exposures, and propose dispositions. It cannot execute a trade, edit a policy, waive a limit, suppress the alert that generated the finding, or mark its own finding resolved. Approved remediation flows through the firm's existing order-management and case-management systems under their normal entitlements and segregation of duties.

That is *Chapter 2*'s rule that irreversible authority is never granted to one role, however trusted, applied to a role that happens to be a model: the model proposes, and a person and an existing system authorize and act. It is stronger than adding "ask a human" to a prompt and more precise than calling the system read-only. Read access can still disclose confidential positions, poison shared memory, consume scarce reviewer attention, suppress a real alert through bad prioritization, or persuade a human to make a harmful trade. Authorization therefore follows the eventual system effect, including human-mediated effects; the verb on the model's tool is insufficient. *Chapter 14* puts the enforcement where this case puts it, in deterministic policy at the effect boundary, with classifiers and filters as detection above it. A fraud deployment the Financial Stability Board (FSB) describes keeps that shape: a large bank's agent proposes detection rules, and the bank's fraud analytics team reviews and approves each one before it runs [FSB, Sound Practices]. The agent has contributed to developing or updating three quarters of the bank's card fraud rules.

The human is a named decision point, not a ceremonial approval box. The reviewer can approve the candidate finding, reject it, request specific evidence, or correct the classification and record why. A portfolio manager may later choose among permitted remediation options, and compliance retains the authority the firm's procedure assigns it. Those actions feed different learning signals: a correction can improve retrieval or interpretation tests; an approval does not by itself prove the recommendation was optimal.

A named decision point can still decay into a click. *Chapter 2* traces how reviewers accept confident wrong output, and prescribes a structured rationale for high-stakes approvals and a sampled audit of approvals so rubber-stamping shows before it becomes the norm; both belong here. The EU AI Act asks the same of the systems it classifies as high risk. An overseer must be able to stay aware of the tendency to over-rely on the system's output, which the Act calls automation bias, and to disregard, override, or reverse that output in any particular situation [EU, AI Act]. Its Annex III financial entries name creditworthiness assessment and credit scoring of natural persons, and risk assessment and pricing in life and health insurance. Portfolio limit monitoring is not among them, so whether the Act reaches this workflow at all

is the classification question *Chapter 14* puts first. The design keeps the override and the guard against automation bias either way, because the failure does not wait for a classification.

Calibrated selective risk can order the candidate findings a reviewer reads, never the engine's alerts, and it cannot transfer accountability to a confidence score. Calibration is measured by policy layer, client type, event type, and consequence tier; coverage is reported beside error. High-scoring findings still pass deterministic evidence and authorization checks, and high-consequence cases may require review regardless of score. Out-of-distribution events and missing source versions escalate and cannot be assigned to a low-priority bucket.

Join traces, evaluation, and release control

The trace reconstructs the event-to-decision path: which rating event arrived, which position snapshot and calculation ran, which documents were retrieved, which model and prompt release produced the candidate finding, what the reviewer saw, and which action the reviewer took. The finding and the reviewer's disposition are the authority record, and *Chapter 12* keeps that record whole in an append-only audit store that sampling never touches, linked to the diagnostic trace when one was kept. FINRA's oversight report suggests the same monitoring to member firms using generative AI: keeping prompt and output logs, tracking which model version was used and when, and having humans review outputs. Its list of risks particular to AI agents reads as a list of this design's boundaries: acting without human validation, acting beyond the user's intended scope and authority, and multi-step reasoning that is hard to trace [FINRA, Oversight Report]. *Chapter 12*'s observability is useful here because *Chapter 11*'s evaluation and *Chapter 15*'s release process read it.

Evaluation compares the candidate system with the baseline on a fixed replay set and then on contemporaneous traffic. It measures false positives, independently ascertained misses, evidence completeness, citation correctness, calibration and selective risk, review time, reviewer disagreement, cost, and queue health. Misses cannot be inferred only from alerts the new system emitted; periodic independent review, incident findings, and the existing rules engine provide outside labels. Slices expose whether aggregate performance hides failures on a jurisdiction, client mandate, security type, or rare policy interaction.

The banking agencies' guidance names what gives such an evaluation its force, even though it leaves generative AI out of its own scope. It describes *effective challenge* as critical analysis by objective experts who have the expertise, the independence, and the organizational standing and influence to effect change. It also says that when a model's design relies substantially on expert judgment, quantitative outcomes analysis helps evaluate the quality of that judgment. The candidate finding is judgment of exactly that kind, the paired comparison with the baseline is its outcomes analysis, and the standing is the fifth question's answer: the evaluation holds the authority to stop a release.

Release proceeds in stages. Offline replay must beat the baseline's declared thresholds. Shadow mode runs on live events without changing the review queue. A canary exposes a small, named portfolio set and stops on hard guardrail breaches, miss-rate regression, evidence failure, or reviewer overload. As in *Chapter 15*, a canary that ends inconclusive holds the release. The release fingerprint binds the model, prompt, retriever, embedding model, corpus snapshot, calculation code, schemas, policy configuration, and tool versions, which is *Chapter 15*'s release manifest with this domain's surfaces named. A hold prevents promotion, and rollback restores the full last known-good bundle.

Lessons learned

Nothing in this case was measured, so each lesson is a design claim to test against the baseline, and between them the five answer the five questions. The claims hold for workflows of this shape, where a deterministic engine already detects the event, the policy corpus keeps a version history, and a reviewer has the authority and the time to decide. They do not carry to an open-ended question such as an investment decision, or to a corpus with no history to replay a finding against. Nor do they fix a topology: separate services, one agent with deterministic tools, or a workflow with a few model calls can each implement the same contracts, so the number of agents is the result of the design, not its premise.

Lesson 1: Draw the line between arithmetic and judgment first, because it is also the line between governance regimes. Under the banking agencies' guidance the design splits three ways. The exposure arithmetic is not a model, the generative model that drafts the finding is outside the guidance's scope, and a non-generative scorer that orders the review queue by calibrated risk is squarely inside it. Each component inherits the control set of its side, so list every component with the regime it falls under before any agent is named.

Lesson 2: Authorize the effect, not the tool's verb. The analysis path is read-only by its tools and consequential by its effects, because a read can still bury a real alert through bad prioritization or persuade a person to make a harmful trade. FINRA's oversight report names the quieter version among the risks of agents on sensitive data, which they may store, explore, or disclose without meaning to, so a read grant needs the same scrutiny as a write.

Lesson 3: Make the finding point at versioned sources, so an audit can replay it. The `CandidateFinding` cites passages, document versions, the position snapshot, and the calculation version, never the model's paraphrase of them. The adviser's five-year retention duty is what makes the replay testable, because "which internal policy governed on the event date" then has a recorded answer. Where two policy layers disagree, the finding records the conflict and cites both, since a blended answer can be replayed but not audited.

Lesson 4: Reviewer attention binds, so score the disposition and audit the approvals. Because the rules engine already detects, an addition can shorten only the time from alert to a

correct, evidenced disposition, and one that raises more candidate findings than reviewers can read moves the bottleneck without widening it. Volume and review quality are one variable: *Chapter 14* warns that a gate that fires often enough teaches the approver to clear it without reading. The FSB's consultation names both halves: real-time human monitoring of agents grows impractical as their use scales, and a reviewer who consistently approves agent recommendations without modification is rubber-stamping them [FSB, Sound Practices]. A review shortened by lowering its quality shows only in the sampled audit of approvals and in the independently ascertained misses, so those two measures sit beside review time and queue depth.

Lesson 5: Give the evaluation standing: it must be able to stop the release. An evaluation without the standing that effective challenge requires only fills a dashboard. The staged release is where that standing bites: a canary that ends inconclusive holds the release, and a rollback restores the whole fingerprinted bundle, not only the model.